DEVELOPMENT OF A HARDWARE AND SOFTWARE SYSTEM FOR POSTURE CORRECTNESS DETECTION

Illia Andrushko
Vitaly Diduk

ABSTRACT

Relevance of the topic. In recent years, there has been an increase in the problem of poor posture among people who spend a lot of time sitting, in particular at the computer. Poor posture can cause physical problems, such as back pain, neck and shoulder discomfort, internal organ disorders, and a deterioration in psychological well-being. There are a number of solutions for maintaining a healthy posture, including gym and exercise, seeing a doctor, orthopaedic corsets and support belts. However, such solutions require additional time and money, which only a small number of people decide to do. In the case of an orthopaedic brace, users also experience additional inconvenience when wearing it on a regular basis. Currently, there are not yet many affordable and perfect solutions for use at home to prevent posture problems, so it is important to develop new technological solutions for recording the correctness of posture and alerting the user in case of prolonged deviations from the norm.

Objective. To develop a hardware and software system for detecting the correctness of posture.

Tasks:

- Analyse the existing means of detecting the correctness of posture;
- find out the normal position of a person when sitting with the correct posture and determine the pressure points on the seat;
- consider and select the necessary sensors, electronic components and materials for the project;
- create a hardware and software system for detecting correct posture;

The object of the work is the process of measuring correct posture using pressure sensors, followed by data analysis and its conversion into text and visual form.

The subject of research is a set of theoretical, technical and practical principles for the creation and implementation of a hardware and software complex for detecting the correctness of posture.

The theoretical and methodological basis of the study is a system of general scientific and special methods, namely: methods of comparison and systematisation, observation,

document study, prototyping and modelling (in the development of software and hardware).

Ways to solve the research problems:

1. Develop a hardware system that will use a universal chair or armchair pad with integrated pressure sensors to collect data on the user's sitting position.
2. Develop software to collect data from the hardware component, analyse it, and generate alerts for the user.
3. Analyse the correct posture of people of different physiques and genders. Based on the collected data, develop a mathematical model of the dependence of the sensor system indicators on the user's position and calculate the degree of deviation from the optimal position. The presented solution has some drawbacks, namely: people who have already acquired some diseases related to their posture will have problems in comparing deviations from the proportions, as they already have a certain curvature of posture and the pressure on each sensor will differ from that of a healthy person.
4. Place the calibration option in the software. In this case, the problem of gender, age, physique and acquired diseases in a person undergoing rehabilitation disappears. After using this function, the software remembers the indicators at the time of calibration for further comparison and notification of the user about his posture at the moment.

Challenges: The initial challenge was encountered during the sensor selection process, which required extensive research of various sources. Additionally, we faced difficulties in selecting the appropriate material for weight distribution on the substrate.The first material chosen proved unsuitable, and the use of operational amplifiers did not yield satisfactory results. Consequently, it opted for a different material (rubber) and abandoned the use of operational amplifiers.The issue of high sensor readings was resolved by implementing resistors with lower resistance. Furthermore, a Bluetooth module was integrated to enable cross-platform connectivity with mobile devices.

Conclusions: The hardware and software complex for measuring the correctness of posture was developed in accordance with the needs of modern society, where most people spend a significant amount of time in a sitting position, which can lead to physical problems. The complex is based on the analysis of pressure points on the seat and comparing their readings with the readings of the sensors at the time of calibration to determine the correct posture.

At this stage, it has been implemented:

-measuring the correctness of posture, namely deviations from the initial position

-graphical visualisation and alerting the user in case of incorrect posture

-30-minute reminder of warm-ups

-The ability to connect the gadget to mobile devices wirelessly

Further ways of development:

-adding a compact vibration drive for tactile interaction with the user

-improving the software part of the device with the help of intelligent systems and neural networks.

-use of this device in the offices of various companies and enterprises whose employees spend a lot of time in a sitting position.

-use of the device in schools with the subsequent sending of information about the child's posture to parents.

Keywords: Hardware and software system, pressure sensors, electronic components, Bluetooth module, operational amplifiers.

TABLE OF CONTENTS

INTRODUCTION…… 6

CHAPTER I. ANALYSIS OF EXISTING MEANS OF DETECTING CORRECT POSTURE…… 8

1.1. Analysis of the research area……...….. 8

1.2. Analysis of existing posture control solutions……... 9

CHAPTER II. HARDWARE AND SOFTWARE IMPLEMENTATION OF THE POSTURE CORRECTNESS DETECTION SYSTEM……... 14

2.1. Development of a system structure diagram…… 14

2.2. Development of system hardware…… 15

2.3. Development of the system algorithm……. 20

2.4. Development of system software…… 22

CONCLUSIONS……..…… 30

LIST OF REFERENCES AND LINKS……... 31

APPENDIX A. Full specifications of the FSR 402 sensor…….. 33

APPENDIX B. Firmware listing of the hardware system……..…….……. 34

APPENDIX C. Listing of software for a personal computer…….…….. 35

APPENDIX D. 3-D and 2-D view in the EASYEDA environment 41

INTRODUCTION

To address this issue, it is important to promote good posture habits and poor posture has become a significant health issue, particularly for those who spend extended periods sitting or working at a computer. It is crucial to maintain good posture to avoid potential health problems.

Modern society is characterised by a significant amount of sedentary work and dependence on information technology. Many individuals spend prolonged periods sitting at desks or in front of computers. This sedentary lifestyle is frequently accompanied by poor posture, which can result in severe health implications.

Poor posture can have a significant impact on a person's health. It can cause physical problems such as back pain, neck and shoulder discomfort, and disorders of the body's organs and systems. Many people who suffer from poor posture experience pain and discomfort, which can significantly affect their quality of life.In addition to the physical effects, poor posture can also have psychological consequences, causing stress and uncertainty and affecting a person's overall mental state. Therefore, it is **important** to develop methods and means of monitoring the observance of correct posture in people working at a desk and computer, including.

Objective. To develop a hardware and software system for detecting the correctness of posture.

Tasks:

- Analyse the existing means of detecting the correctness of posture;
- find out the normal position of a person when sitting with the correct posture and determine the pressure points on the seat;
- consider and select the necessary sensors, electronic components and materials for the project;
- create a hardware and software system for detecting correct posture;

The object of the work is the process of measuring correct posture using pressure sensors, followed by data analysis and its conversion into text and visual form.

The subject of research is a set of theoretical, technical and practical principles for the creation and implementation of a hardware and software complex for detecting the correctness of posture.

The theoretical and methodological basis of the study is a system of general scientific and special methods, namely: methods of comparison and systematisation, observation, document study, prototyping and modelling (in the development of software and hardware).

CHAPTER I

1.1 Analysis of the research area

The problem of poor posture requires a comprehensive approach. One possible solution is to develop and implement specialised devices and systems aimed at monitoring posture. These devices may include a posture measurement and correction system that can detect poor posture and alert individuals to the need to correct it. There are various solutions to this problem, including regular:

-**Exercise and training** to strengthen muscles, improve posture, and alleviate back problems.

- **Seeking advice from an orthopaedic surgeon** or physiotherapist can also provide individualised recommendations for correcting posture and addressing back issues.

-**Orthopaedic braces and support belts** may be helpful. Individuals may use orthopaedic braces or support belts to improve posture and reduce discomfort. It is important to note that the use of these devices should be discussed with a healthcare professional to ensure proper fit and effectiveness.

The selection of a posture correction method may depend on individual needs and the recommendations of a specialist.

However, few individuals have the desire, time, and financial resources to regularly attend a gym. Visits to a doctor for preventive care and consultations are also infrequent, and individuals typically seek such visits only when they require direct treatment for acquired problems. For many users, the optimal solution is to utilise orthopaedic chairs. However, this tool may not always provide accurate results if individuals do not sit in compliance with the relevant requirements. As an alternative, the paper proposes the development of a hardware and software complex that can monitor a person's posture in any chair.

1.2. Analysis of existing posture control solutions

Currently, there is a relative lack of publicly available solutions for posture control and correction, despite the urgency and relevance of the issue. However, the variability and multifunctionality of the available solutions are noteworthy and can be useful for users seeking to maintain healthy posture and prevent potential medical problems. It is important to note that existing solutions also have limitations and drawbacks. Most of the available options for monitoring and correcting posture may be underdeveloped or incomplete.

Orthopaedic chairs and seats are technologically advanced solutions designed to ensure a comfortable and healthy posture. They are made using special materials and designs that help maintain the correct body position. However, it is important to note that these orthopaedic solutions cannot measure correct posture in real-time and do not provide a fixed posture, allowing users to sit in different positions. This feature can provide a range of body positions, but it may not always maintain correct posture due to the lack of automatic control.



Fig. 1.1. ERGOHUMAN ELITE 2 orthopaedic chair

Today, we can distinguish between two types of orthopaedic chairs: orthopaedic chairs of the usual structure, where the main parts of the structure are the backrest and seat (Fig. 1.1), and special orthopaedic knee chairs (Fig. 1.2).

The specific model shown in the picture is the ERGOHUMAN ELITE 2. This model has the ability to adjust the seat height, seat depth (by 50 mm), backrest angle (locked or free mode), synchronous rocking mechanism, backrest height (6 positions), separately adjustable lumbar support, adjustable headrest (2D) and armrests (5D) that can be raised for working with a smartphone.

The chair is designed with an ergonomic mesh that has 10 zones of varying tension for optimal comfort, thermoregulation, and weight distribution. The mesh is made of nylon and cotton weaves and can support up to 120 kg. The chair's frame is made of FPS plastic (functional polymer composite), while the base and crosspiece are made of cast aluminium.

However, like all orthopaedic chairs, this model has the disadvantage of not ensuring correct posture while sitting, as it allows you to sit in an arbitrary seat shape. Additionally, the high price of this product is 700USD.



Fig. 1.2 Kneeling chair

In the second half of the 20th century, Danish Dr Mendel noticed that children who sit in a chair for a long time eventually begin to fidget, trying to change their posture. At the same time, most of them slide to the very edge of the seat and tilt their bodies forward,

shifting the load from the pelvis to the knees and legs. The spine bends slightly in the lower back and relaxes. As a result of these observations, he created an orthopaedic chair with a 15° seat. Sitting on it was not very comfortable - the body would slip and he had to return it to its original position all the time. In parallel with Denmark, work on the design of furniture that is comfortable for long periods of sitting was carried out in Norway. The talented furniture designer Peter Opsvik came up with the idea of installing a special knee bar on the chair. This made sitting more comfortable by fixing the body in a comfortable position and preventing it from slipping. This is how the kneeling chair was born.

The first results of using a kneeling chair become noticeable in 3-4 weeks: sleep and appetite improve, performance increases, the digestive and respiratory systems improve, and pain in the lower back and knees disappear. However, in case of serious diseases of the spine and joints (extreme deformation of the spinal column, bursitis, dislocation of the hip), an orthopaedic knee chair can provoke an exacerbation, which is a disadvantage of this product.

Despite the existing classic solutions, the world is not standing still. We live in the 21st century - the century of technology, so Chinese engineers from Xiomi came up with the idea to create an electronic posture corrector, which works by measuring the angle of the body using an accelerometer built into the device. The device itself looks like an open ring that should be worn around the neck so that the main part of the device is on the back (Fig. 1.3).



Fig. 1.3 Xiaomi Hipee P1 Posture Corrector

The Hipee P1 corrector utilises real-time sensors to track incorrect spinal position, raised and lowered shoulders.

Unlike traditional posture correction belts (Figure 1.4), this gadget constantly reminds the user to use their own muscles to maintain proper posture. The gadget's creators suggest that regular muscle training is a more effective approach to solving this problem. The device can detect various human postures, including sitting, bending, writing, and standing, to avoid excessive reminders that may disturb the user.

It provides a silent vibration reminder through the posture correction module when a person's spine deviates from a range of 5-20° until the correct posture is restored. The WeChat app also displays real-time changes in posture. If the angle of your posture exceeds the set limit, the posture corrector indicator will turn red, and a warning will appear in the app. The Xiaomi Hipee P1 smart posture corrector monitors the user's cervical spine in real-time, reminds them to stretch their muscles, and provides engaging exercises to help form the habit of sitting and walking with correct posture.

However, this device has a significant disadvantage as it is inconvenient and impossible to use while walking or running fast. The corrector has a flexible rubber base, which makes it likely to fall off and get damaged. It is best to use it when standing still or sitting for a long time. Additionally, the device only measures the angles of the upper spine and shoulders, and is not able to measure pelvic displacement in a sitting position. Additionally, the device may lead to circulatory issues in certain individuals.

Wearing a posture corrector (Fig. 1.4) allows you to gradually retrain your muscles to maintain a correct posture.The corrector helps reduce discomfort by providing support to these areas.If you choose the wrong corrector, or if you use it on your own without medical advice, you may be harmed by it. With constant use, the muscles become even weaker, which leads to increased pain, spinal curvature, and other diseases of the musculoskeletal system.



Fig. 1.4 Posture corrector

CHAPTER II
DEVELOPMENT OF A HARDWARE AND SOFTWARE SYSTEM FOR DETECTING THE CORRECTNESS OF POSTURE

2.1 Development of a system structure diagram

The stage involves the selection of pressure sensors that can read the force of pressure on the device at 4 different points while a person is sitting, materials to ensure user comfort and correct operation of the device, and a microcontroller with the power to provide the necessary primary processing of the data obtained on a person's posture in a sitting position while using this device, development of an electrical circuit diagram, writing firmware for the microprocessor and development of the software code.

The software part of the system provides for the processing and storage of the received information from each of the four sensors for processing and analysis, which will allow you to check the correct posture in the future use of the device and visualise the information on the computer screen. Import of information and power supply of the microcontroller will be carried out via USB - a mini USB cable connected directly to the data output device. Based on the data obtained, it is possible to check the correctness of posture in the future. The software constantly checks the stored data during the calibration of the device with the current sensor readings.

When a person sits on the device, which is a kind of substrate that allows them to use it on an arbitrary chair or stool, it is necessary to take into account the distribution of weight on its surface, so it is necessary to select materials that will cover the sensors and create a surface for sitting. It is also advisable to use two layers of this material, one layer of which will be located under the sensors and the other above them, and the material itself should be soft and elastic at the same time to distribute the load.

The block diagram of the device operation is shown in Fig. 2.1



Fig. 2.1 Block diagram of the device operation

2.2 Development of system hardware

For this project, we used FSR 402 pressure force sensors (Figure 2.2), which are part of the single-zone family of force sensing resistors. Force sensing resistors, or FSRs, are rugged polymer thick film (PTF) devices that exhibit a decrease in resistance as the force applied to the sensor surface increases. This force sensitivity is optimised for use in the sensor control of electronic devices such as automotive electronics, medical systems, and industrial and robotic applications. The standard FSR 402 sensor has a circular shape with a diameter of 18.28 mm. Full specifications of this element are presented in Appendix A.



Fig. 2.2 Force sensor FSR 402

4 sensors are used for the correct operation of the device. Each sensor together with a constant resistance is connected in series. The end points are connected to the 5V and Gnd power lines. The centre point is connected to the analogue input of the Arduino Nano. When connected in series, the voltage will be distributed between these two resistors.

Since the resistance of the FSR 402 changes when the force of pressure on it changes, the voltage at the centre point will also change.

An open source microcontroller breadboard based on the ATmega328P microcontroller - Arduino Nano was used to read the electrical signal from the sensors and then transmit the information to the devices for processing the information and implementing the software algorithm (Fig. 2.3). This model was chosen because of its functionality and compactness.

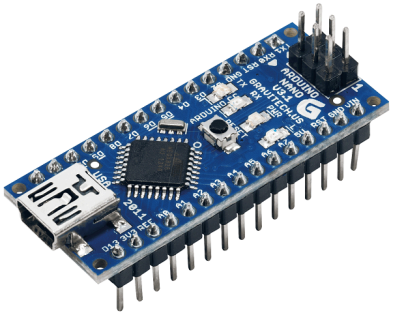


Fig. 2.3 Arduino Nano

The microcontroller together with pre-soldered sensors is connected to the data output device via a USB - mini USB cable for power supply and further communication with a computer.

It is also a reasonable solution to use a Bluetooth module, which allows for wireless monitoring and enables the use of smaller posture monitoring devices such as a phone or tablet. In this development, the HC-05 module (Fig. 2.4) was used, which is an easy-to-use Bluetooth SPP (Serial Port Protocol) module designed for transparent wireless serial connection. The HC-05 Bluetooth module can be used in a Master/Slave mode, making it an excellent solution for wireless connections.

Specifications:

Bluetooth chip: HC-05 (BC417143)

Radio frequency range: 2.4-2.48 GHz

Transmission power: 0.25-2.5 mW

Sensitivity: -80 dVm

Power supply voltage: 3,3-5 B

Current consumption: 50 mA

Range: up to 10 metres

Interface: serial port

Modes: master, slave

Dimensions: 27 x 13 x 2.2 mm.

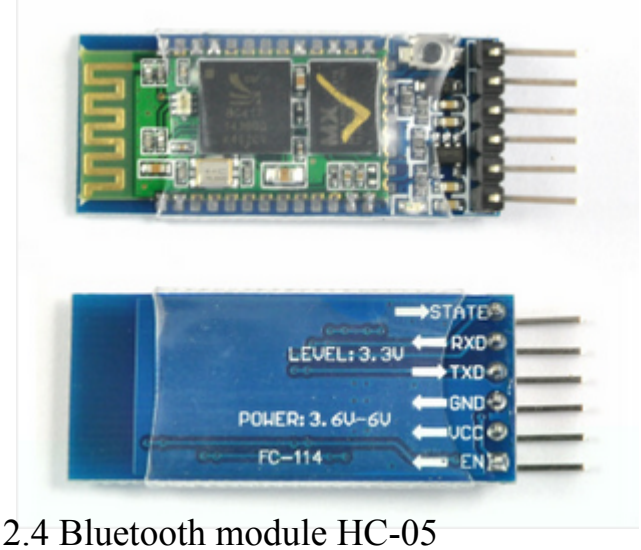


Fig. 2.4 Bluetooth module HC-05

The wiring diagram of the hardware and software complex for detecting the correctness of posture is shown in Fig. 2.5. All components were placed on the small breadboard (Fig. 2.6).

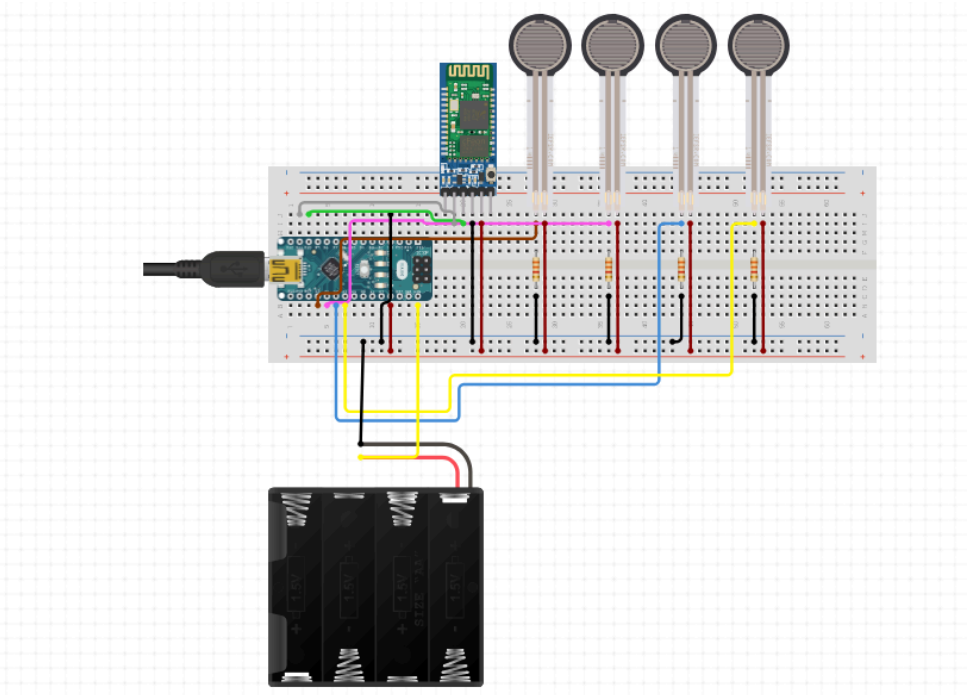


Fig. 2.5 The wiring diagram of the components of the hardware and software system for detecting correct posture

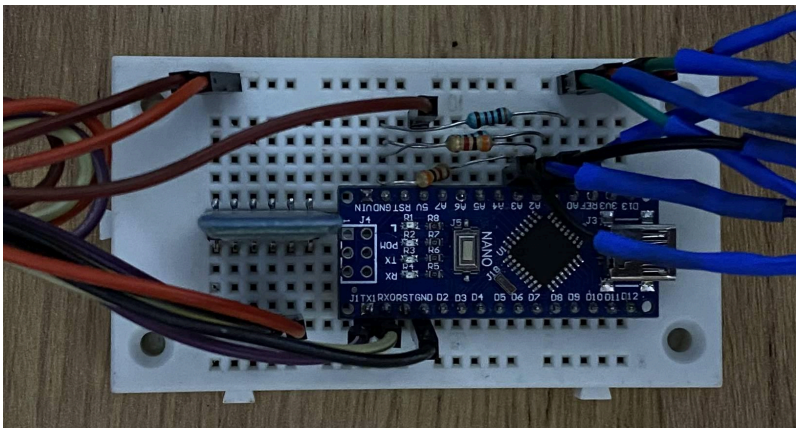


Fig. 2.6 The breadboard with the components soldered on it

To distribute the weight at each pressure point during the seat, as already mentioned, a soft and elastic rubber material (Fig. 2.7), which is commonly used for noise insulation in cars, was used. In total, it was used 2 layers of 5 mm thick and 32x32 cm in size.



Fig. 2.7 Rubber material

However, the sensors were positioned on the surface of the rubber material in the usual way (with each sensor offset by 5 cm from the end in two coordinates), and thus the indicators were subject to significant distortion by an error with a relatively small movement of the user. In light of these observations, the decision was taken during the course of the device testing phase to divide the surface area into four quadrants for sensor

interrogation, employing both software and hardware. Each quadrant consisted of a sandwich structure comprising two square plywood plates and a layer of rubber material with a sensor positioned between them (see Fig. 2.8). This configuration permitted the creation of a more robust foundation for the individual sensors, which in turn compensated for irregularities in the seat surface and the unique physical characteristics of users, thereby facilitating more precise readings of the individual sensors. Figure 2.9 illustrates a comparative representation of the alteration in sensor readings over time during a sitting posture with optimal alignment, with the first type of construction and the second.

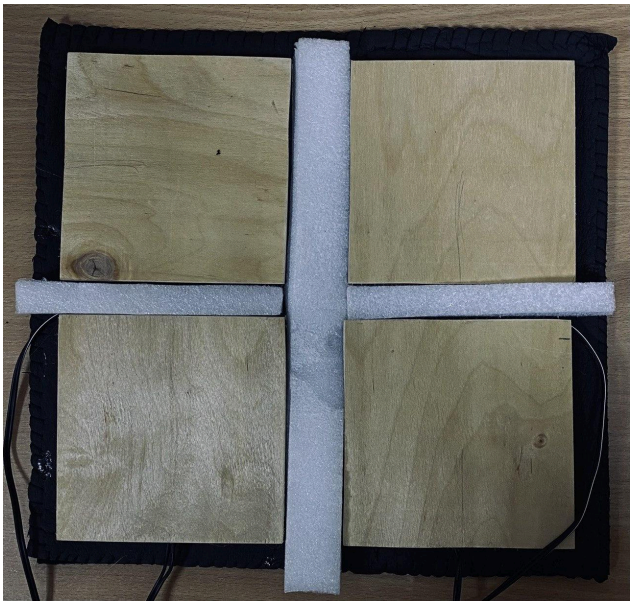


Fig. 2.8 Internal view of the device

Created by the author

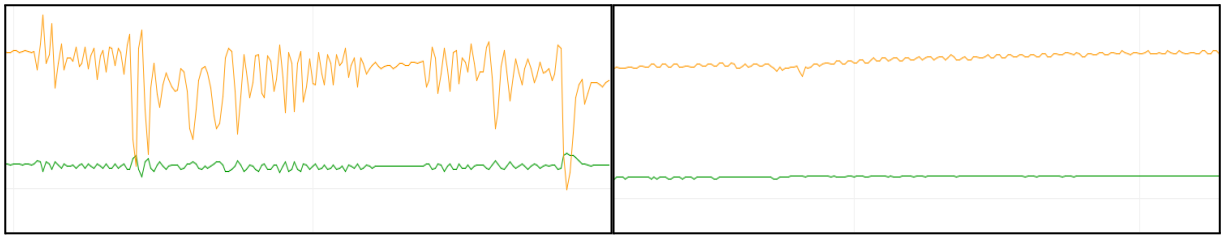


Fig. 2.9 Comparative characteristics of changes in sensor readings over time during sitting with correct posture with the first type of construction and the second.

Created by the author

After the stage of hardware and software testing, an electrical circuit diagram (Fig. 2.10) and the corresponding printed circuit board were developed in the EASYEDA environment (Fig. 2.11).

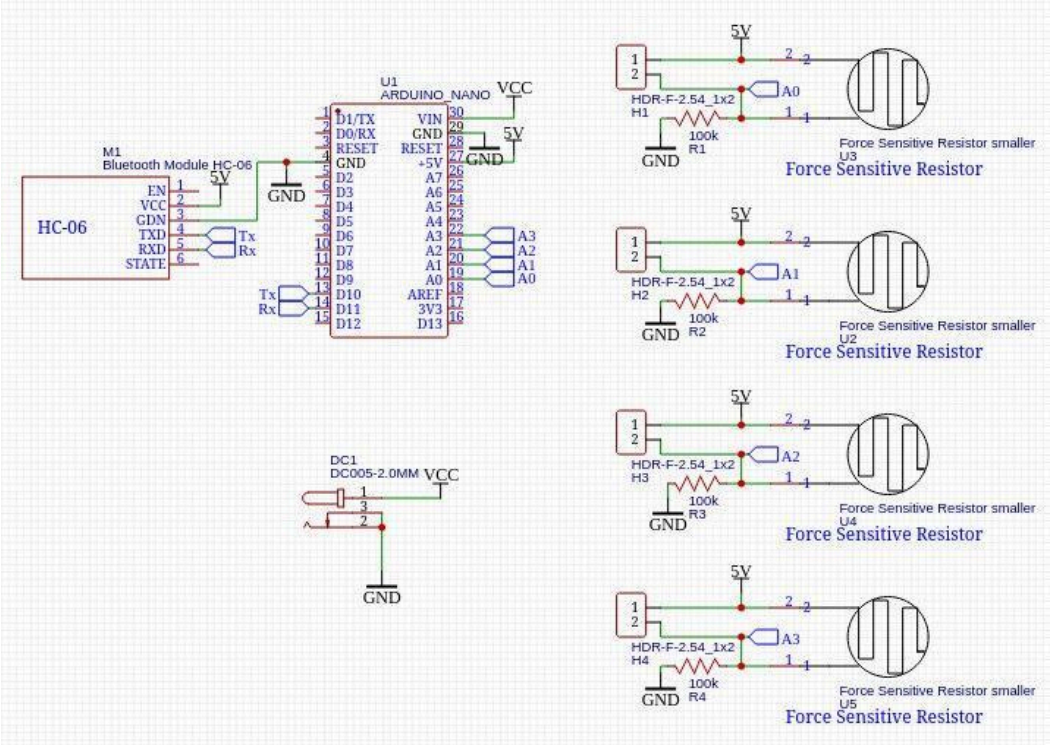


Fig. 2.10. Electrical schematic diagram of the hardware and software complex

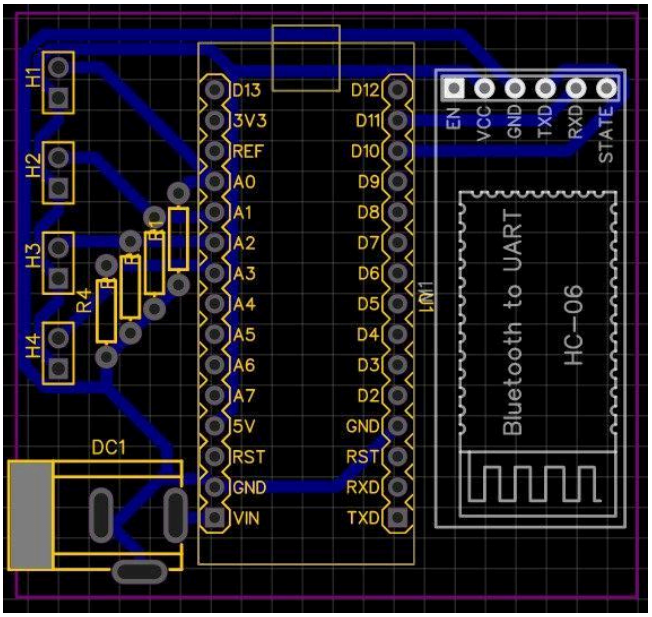


Fig. 2.11. Model of a printed circuit board in the EASYEDA environment

The appearance in the 3-D viewing mode in the EASYEDA environment is presented in Appendix E.

On the basis of the designed PCB, a printed circuit board of foil textolite was manufactured using the laser-iron method. The holes for mounting the components were drilled with a hand drill. After the components were installed, the circuit was soldered using a soldering station, solder and flux. To ensure autonomous operation, the device was equipped with rechargeable batteries and a charge/discharge controller. After checking the system for functionality, the electronics were mounted in a plastic case, which completed the device.

2.3 Development of the system algorithm

Upon commencement of the program, the microcontroller initiates the preliminary parameter values and configures the peripherals for input/output data processing. In the main cycle of the program, the signal value of each sensor is transmitted to a computer or mobile device, depending on the connection method, with the data pre-filtered by the "running average" principle. The user is able to observe the readings of each sensor in the application window in the form of a graph demonstrating the dependence of the sensor values on time and the visualisation of the device (Fig. 2.12).

Once the correct sitting position has been assumed (with the shoulder blades together, the back fully against the back of the chair, and the lumbar curve not excessive, while the knees are bent at an angle of slightly over 90 degrees), the user must then select the "calibrate" option via a dedicated button on the main screen of the program. Upon pressing this button, the program will store the initial position of the centre of mass as a reference point. The values obtained from the strain gauge are placed on the quadrants of the coordinate grid, with these indicators used as the length of the conditional vector. The sign changes depending on the quadrant. The centre of mass is defined as the point of intersection of the diagonals between quadrants 1, 3 and 2, 4. A median filter was applied

to the signal that transmits the coordinate values responsible for the movement of the centre of mass in the coordinate grid, with the objective of reducing random disturbances within a certain interval.

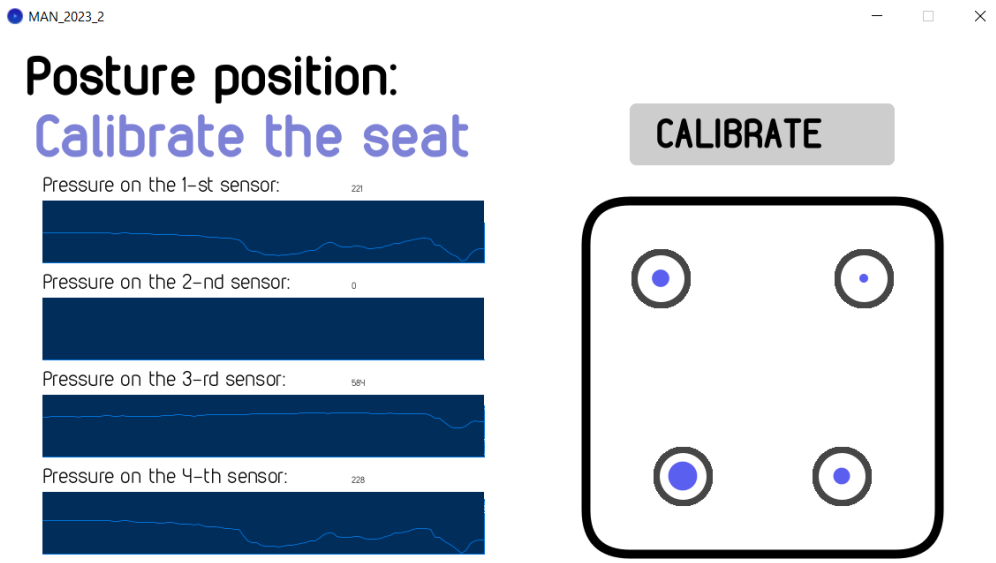


Fig. 2.12 View of the application for visualising the received information and alerting the user

Should the deviation of the centre of mass exceed 80% of the initial absolute position within a specified time frame, the program will indicate that the posture is incorrect.

As soon as the person takes the correct position again, the app will stop notifying the user.

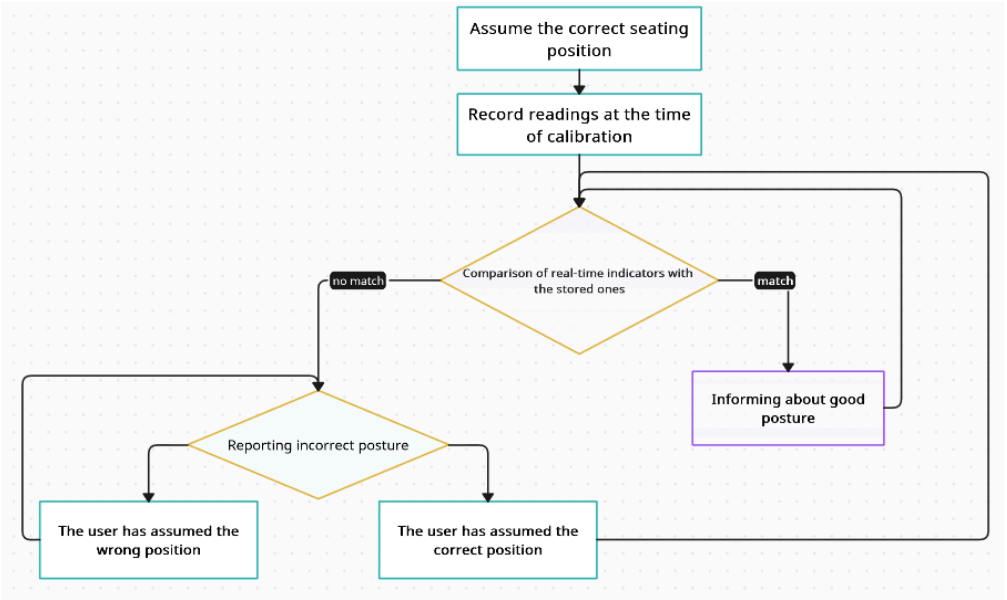


Fig. 2.13 Software operation algorithm

2.4 Development of system software

The project involves the creation of software and hardware for receiving data from pressure sensors, transferring this data to an application installed on a computer or laptop using a USB cable. It also involves the creation of an application in a mobile phone with similar functions using Bluetooth wireless technology, which makes this development cross-platform.

To firmware the Arduino Nano microcontroller, that is, to load the code for reading and outputting data from the Arduino analogue ports (namely A0-A4) into its memory, we used the official firmware program through the communication ports of the Arduino IDE (Fig. 2.14).

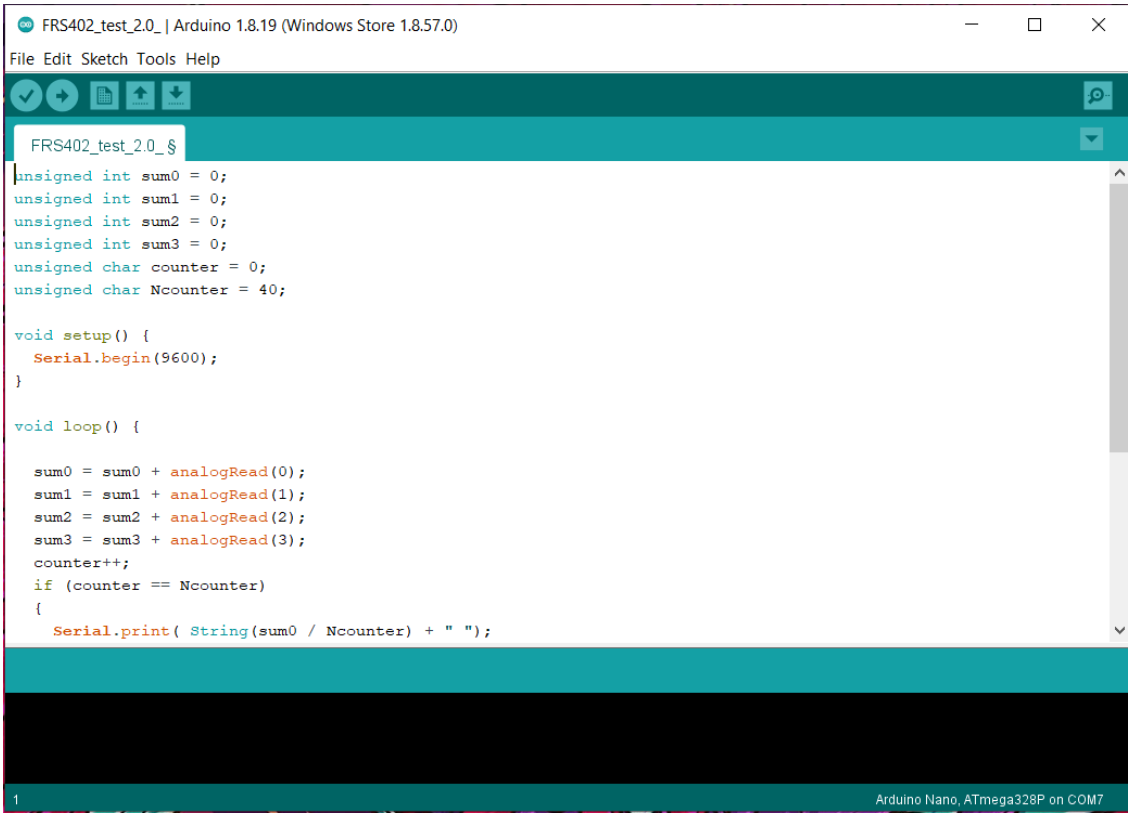


Fig. 2.14 The Arduino IDE application together with a part of the code for flashing

The Arduino IDE is a multi-platform Java application that includes a code editor, compiler and a module for transferring firmware to the board. The development environment is based on the Processing programming language and is designed for

programming by beginners who are not familiar with software development. The programming language is similar to the Wiring language. In general, it is C++, supplemented by some libraries. Programs are processed using a preprocessor and then compiled using AVR-GCC. Arduino programs are written in the C or C++ programming language. The Arduino development environment comes with the Wiring library (originating from the Wiring project, which allows you to perform many standard I/O operations much easier).

The firmware code provides for reading the signal from the analogue ports, transmitting them to the computer and displaying the readings in the terminal. The full code for the Arduino IDE is presented in Appendix B.

The Processing software environment was used to create the information visualisation application itself (Figure 2.15). Processing is a free graphical library and integrated development environment (IDE) designed for electronic art, new media, and visual design to teach non-programmers the basics of computer programming in a visual context. Processing uses the Java programming language, with additional simplifications such as additional classes and aliased mathematical functions and operations. The environment also includes a graphical interface to simplify the compilation and execution phase.

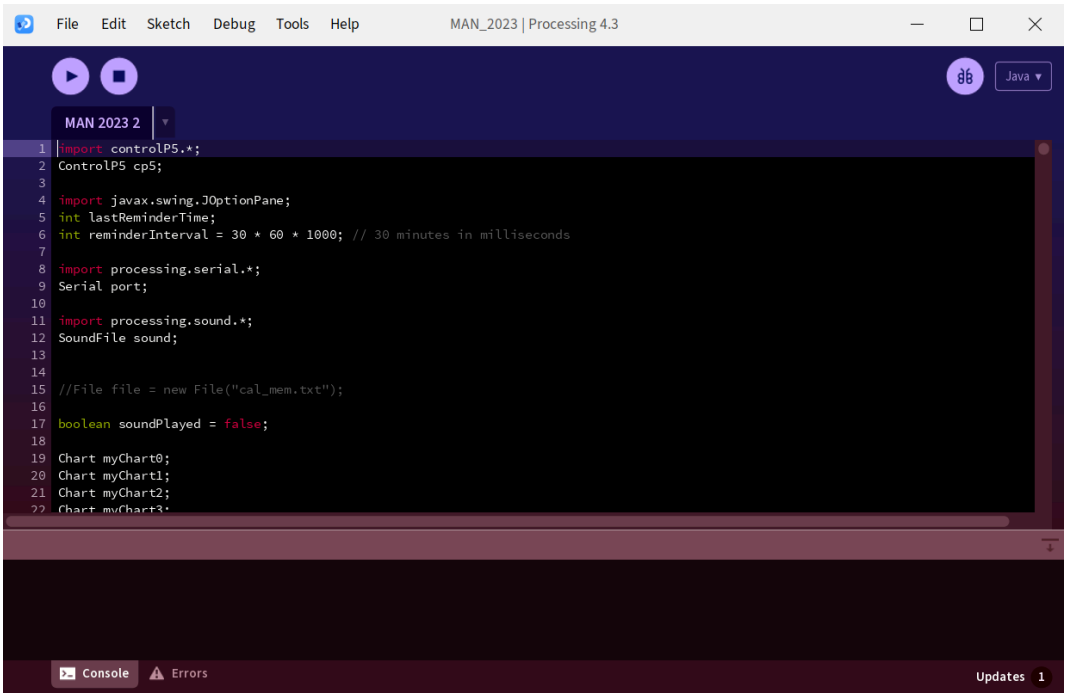


Fig. 2.15 The appearance of the Processing software environment

The Processing language and IDE was the precursor to other projects including Arduino, Wiring, and p5.js. It is used by students, artists, designers, researchers, and hobbyists to explore prototyping and manufacturing. It was created to teach the basics of computer programming in a visual context and serves as a software (meaning that each visual *.pde file of the Processing shell is represented by a separate image or animation, etc.) and a professional tool. It is also possible to export the programme to an *.exe file to run it on computers that do not have this software environment.

The first step in the development of the application was the development of the interface (see Fig. 2.12). The application window is 1142 by 624 pixels in size, with information about the current posture position at the top of the window, and 4 different text options can be displayed in this field:

-"Calibrate seat" (when the calibration button has not yet been pressed)

-"Good posture :)" (when the user has the correct posture)

-"Sit up straighter on the seat :|" (when the user does not maintain the initially correctly calibrated posture)

-"Sit on the seat" (when the user has stood up from the device)

The left side of the window contains diagrams that show the user the force of pressure on each sensor. To implement such diagrams, the additional library controlP5.* was used.

ControlP5 is a graphical user interface (GUI) library for the Processing programming environment. It provides a set of GUI components that you can use to create interactive interfaces for your sketches in Processing. The library is especially useful when you need to add buttons, sliders, text boxes, and other interactive elements.

It is also advisable to visualise the device itself in the application, which is located on the right side of the window. This visualisation is a square with rounded edges in the middle of which there are four circles representing the four sensors located on the device itself. In the centre of each circle is a blue circle, the radius of which changes when the values of the sensors change.

The last part of the visualisation is the button with the inscription "CALIBRATE", which is pressed to calibrate the device.

Also, when a person's posture deteriorates, the application not only visually notifies about it, but also notifies about it with a sound signal. To use sound files and play them in the application, the processing.sound.* the library was used.

The second step in developing an application is to write a function with specific lines of code for information containing sensor readings from the computer port to which the Arduino is connected. To do this, use the processing.serial.* library, which allows you to receive information from computer ports and use it in the software environment.

To address the issue of prolonged sitting, it is recommended to set a reminder for a 5-minute warm-up or exercise. A pop-up notification with a timer and reminder will appear every half an hour (Fig. 2.16). This will help maintain the correct posture and prevent muscle strain.

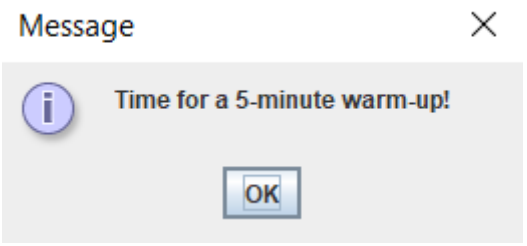


Fig. 2.16 Pop-up notification with a warm-up reminder

To implement such a notification in the selected software environment, the javax.swing.JOptionPane library is used, which allows you to create various pop-up messages.

The entire program code is shown in Appendix C.

To develop the application for mobile phones, it was chosen the visual android application development environment App Inventor. App Inventor uses a graphical user interface for programming, a visual programming language very similar to Scratch. The language aims to teach children the concept of programming and allows them to create applications by building the interface and logic for the application using blocks.

The programming environment can be used free of charge in school or extracurricular education to introduce children to the programming base. The main

purpose of the application is to display the user's posture position, push notifications and the ability to calibrate the device.

The first step in developing the application was to design the interface. The interface of mobile applications plays a very important role in the process of creating an application, because the interface is the link between the hardware and software of a mobile device and the focus of user interaction. To develop the interface, the App Inventor software environment provides a convenient designer with basic elements, markup, and many more functional elements for building various applications.

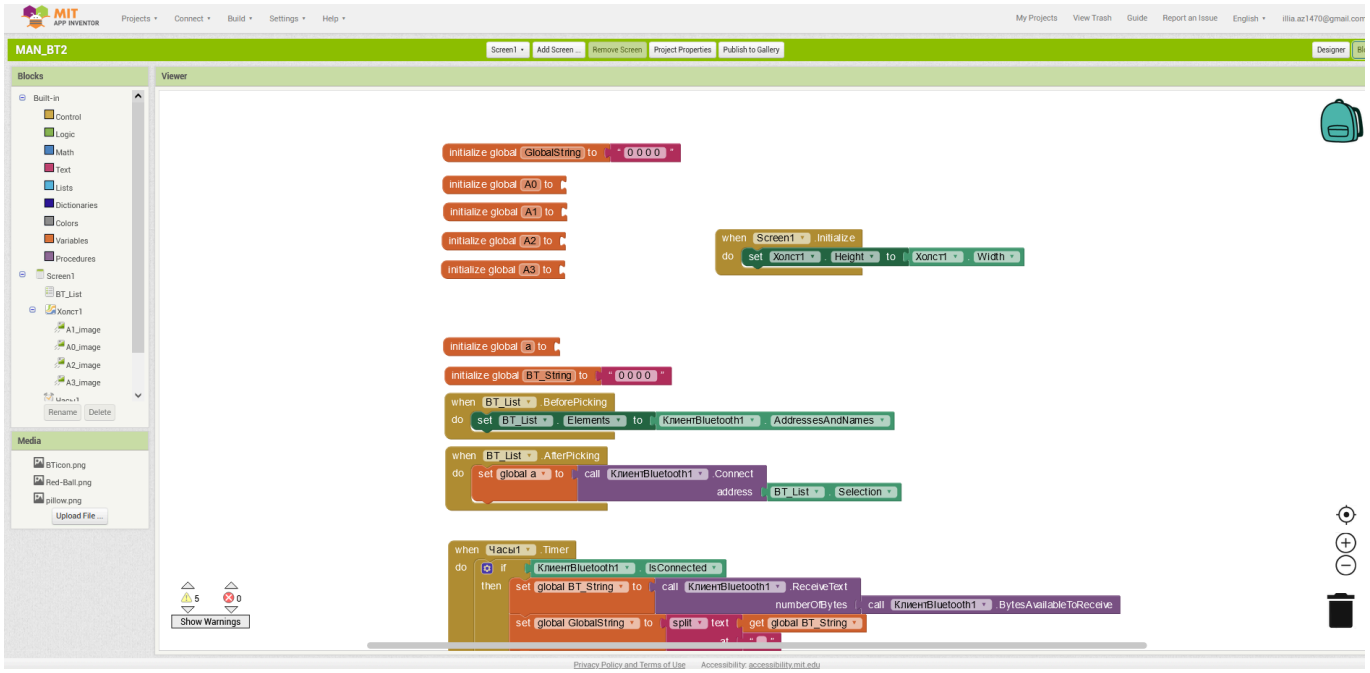


Fig. 2.17 App Inventor software environment

All the elements have basic settings, so with the help of property settings for each element, we give it properties and location on the smartphone screen.

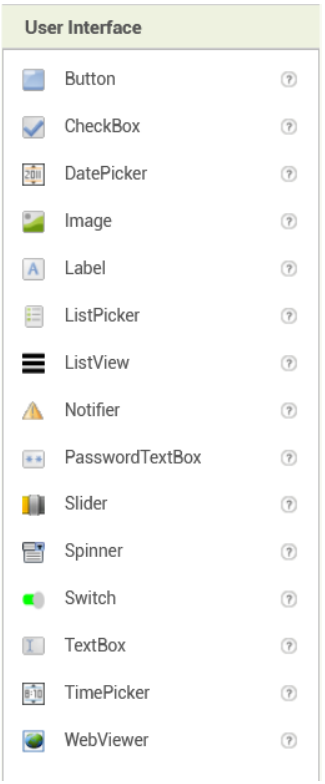


Fig. 2.18. List of elements for building an interface



Fig. 2.19 Setting element properties

When building the interface, it was a problem that the application looks different in the software environment and on smartphones.

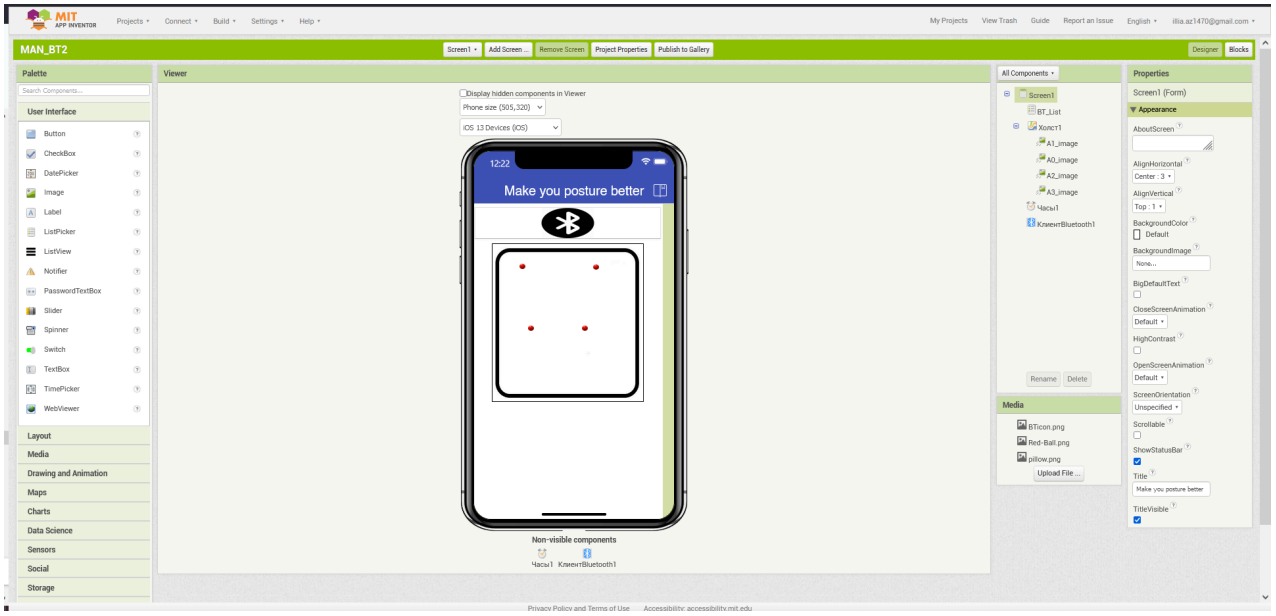


Figure 2.20 View of the application graphical interface in the software environment

However, to test the display on smartphones, the software environment provides a quick download of the application to the device. Using the App inventor app, we scan the QR code and test the app on any device.

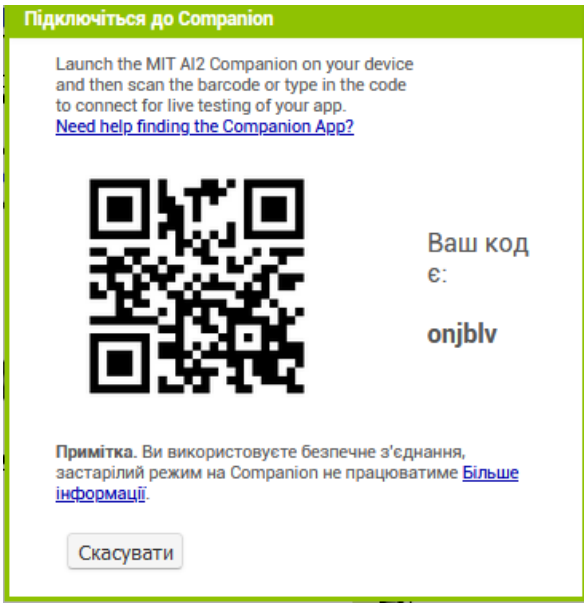


Fig. 2.21 QR code view for receiving the application

The data source is the data obtained from the pressure sensors installed in the device.

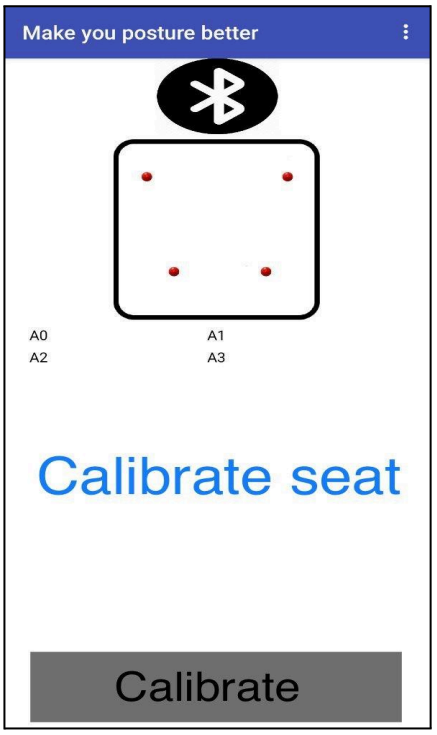


Fig. 2.22 The graphical interface of the application on a smartphone

We can implement everything else using the App Inventor visual android application development environment. This environment allows you to process data using logical blocks and display it to the user. The logic of the program is reflected and described in the appendices of the paper (APPENDIX D).

Chapter 2: Conclusions

This chapter considers the components used, the software environments for application development, and the algorithm of the hardware and software complex for detecting posture correctness. The outcome of both programs is the acquisition of data from sensors, the processing of said data, and the display of all sensor readings. Additionally, the programs possess the capability to calibrate for correct posture.

CONCLUSIONS

The problem of poor posture is relevant and significant for modern society due to its potentially harmful impact on physical health. Solving this problem is becoming an increasingly important task, and research in this area has great potential to improve the quality of life and health of people and requires innovative approaches, such as the development of devices and systems for monitoring and correcting posture.

As a result of the research and analytical tasks of the work, the existing methods of posture control and determination of the parameters of correct sitting were analysed. The development includes the selection of sensors and components to create a complex designed to accurately measure and visualise posture indicators. The study is based on a fundamental theoretical approach and uses advanced technical solutions, which makes it relevant and important for further research in this area.

In further development of the study, the use of artificial intelligence to analyse the data and provide individual posture correction recommendations could also be part of further research. It is possible to apply the development in schools and workplaces, offices using servers and then sending statistics and analyses to the mobile devices of parents or employees of business offices. An important aspect is the study of the impact of corrective exercises and ergonomic equipment on improving posture. This will help to develop a comprehensive approach to the prevention and treatment of postural deviations, contributing to overall health improvement.

After completing the UF StartUp School Beginner course and receiving feedback and recommendations at the I-Fest international competition in Tunisia, the project is planned to be upgraded to a start-up with stakeholders involved in its financing and

production. It is planned to find sponsors for the production of small batches of products to be donated to schools, as such institutions have medical staff and the experiment can take place in a controlled environment under the supervision of a medical professional. Such an event would provide an opportunity to improve the focus group's condition and get real feedback on the effectiveness of the system.

LIST OF REFERENCES AND LINKS

1. Arduino Nano. (n.d.). Arduino Store. https://store.arduino.cc/products/arduino-nano
2. Force Sensitive Resistor (FSR) 402. (n.d.). Arduino.UA. https://arduino.ua/ru/prod242-datchik-davleniya-dlya-arduino-ot-sparkfun
3. FSR 402 Data Sheet. (2010, October 26). Trossen Robotics. https://www.trossenrobotics.com/productdocs/2010-10-26-DataSheet-FSR402-Layout2.pdf
4. Grove - Round Force Sensor FSR402. (Last updated on Jan 19, 2023 by jianjing Huang). Seeed Studio Wiki. https://wiki.seeedstudio.com/Grove-Round_Force_Sensor_FSR402/
5. Photocell Resistor Connection Diagram [39:43]. (2022, April 29.). YouTube. https://www.youtube.com/watch?v=hPZBRTszNj4&t=2383s
6. Velcro Loop Tape 20 cm. (n.d.). Prom.ua. https://prom.ua/p1673474650-velkro-polotno-petelnoe.html
7. Samsung INR18650-20R 2000mAh Battery. (n.d.). 18650 Charger. https://18650charger.com.ua/p1426907564-akkumulyator-samsung-inr18650.html
8. Processing Reference. (n.d.). Processing. https://www.processing.org/reference
9. Arduino and Processing. (n.d.). Arduino Playground. https://playground.arduino.cc/Interfacing/Processing/
10. Adding a Chart to the Application Window [32:40-35:40]. (2021, April 30). YouTube. https://www.youtube.com/watch?v=IfWxl5LhJE8
11. Control Arduino Using GUI (Arduino + Processing). (2017, July 21). YouTube. https://www.youtube.com/watch?v=5WjEQSMiqMQ
12. MIT App Inventor. (n.d.). Massachusetts Institute of Technology. https://appinventor.mit.edu/
13. Knee Chair vs. Regular Chair: Which is Better? (2021, April 23). AMF.ua. https://amf.ua/ua/news/kolennyjj_stul_protiv_obychnogo_kakojj_luchshe-361/
14. Xiaomi Hipee P1 Posture Corrector. (n.d.). Touch. https://touch.com.ua/item/korrektor-osanki-xiaomi-hipee-p1-posture-corrector/?__cf

_chl_tk=u2CslUlnOzaKbIEMFsfn9H0KrOUrMyqVtrLMybuamps-1695481939-0-gaNycGzNC3s

15. ERGOHUMAN ELITE 2 Chair (EHE2-AG-HAM-5D-L, Mesh T-168-B2 Natural). (n.d.). Kreslalux.ua. https://kreslalux.ua/kresla-rukovoditelya/2310-kreslo-ergohuman-elite-2-ehe2-ag-ham-5d-l-setka-t-168-b2-natural-ergonomichnoe
16. Creately - a site for creating flowcharts. (n.d.). https://app.creately.com
17. EasyEDA. (n.d.). Web-based EDA tool suite. https://easyeda.com/

APPENDICES

APPENDIX A. Full specifications of the FSR 402 sensor.

Device Characteristics

Feature	Condition	Value*	Notes
Actuation Force		0.1 Newtons	
Force Sensitivity Range		0.1 - 10.0^{2} Newtons	
Force Repeatability[3]	(Single part)	± 2%	
Force Resolution[3]		continuous	
Force Repeatability[3]	(Part to Part)	±6%	
Non-Actuated Resistance		10M W	
Size		18.28mm diameter	
Thickness Range		0.2 - 1.25 mm	
Stand-Off Resistance		>10M ohms	Unloaded, unbent
Switch Travel	(Typical)	0.05 mm	Depends on design
Hysteresis[3]		+10%	$(R_{F+} - R_{F-})/R_{F+}$
Device Rise Time		<3 microseconds	measured w/steel ball
Long Term Drift		<5% per $\log_{10}$(time)	35 days test, 1kg load
Temp Operating Range	(Recommended)	-30 - +70 °C	
Number of Actuations	(Life time)	10 Million tested	Without failure

APPENDIX B. Firmware listing of the hardware complex

```
#include <SoftwareSerial.h>
SoftwareSerial mySerial(10, 11); // RX, TX

unsigned int sum0 = 0;
unsigned int sum1 = 0;
unsigned int sum2 = 0;
unsigned int sum3 = 0;

unsigned char counter = 0;
unsigned char Ncounter = 60;

int sensor1 = 100; // Value for the first sensor (1st quadrant)
int sensor2 = 150; // Value for the second sensor (2nd quadrant)
int sensor3 = 200; // Value for the third sensor (3rd quadrant)
int sensor4 = 250; // Value for the fourth sensor (4th quadrant)

// Variables for storing the coordinates of points in each quadrant
int x1, y1, x2, y2, x3, y3, x4, y4;

// Center of mass for diagonals
float centerX_diag1, centerY_diag1;
float centerX_diag2, centerY_diag2;

// Center of mass at the point where the diagonals intersect
float centerX, centerY;

// Arrays to store the last three centerX and centerY values for the median filter
float centerX_values[3] = {0, 0, 0};
float centerY_values[3] = {0, 0, 0};

// Index to store the last values
int filterIndex = 0;

// Function for sorting three elements and returning the median using a ternary operator
float medianFilter(float values[]) {
  float a = values[0];
  float b = values[1];
  float c = values[2];

  // Median filter using ternary operators
  return (a > b) ?
```

```
          ((b > c) ? b : (a > c ? c : a)) :
          ((a > c) ? a : (b > c ? c : b));
}
void setup() {
  Serial.begin(9600);
  mySerial.begin(9600);
}

void loop() {
  sum0 = sum0 + analogRead(0);
  sum1 = sum1 + analogRead(2);
  sum2 = sum2 + (analogRead(5)) / 2;
  sum3 = sum3 + analogRead(7);
  counter++;

  if (counter == Ncounter) {
  sensor1 = sum1 / Ncounter;
  sensor2 = sum0 / Ncounter;
  sensor3 = sum2 / Ncounter;
  sensor4 = sum3 / Ncounter;

// Define the coordinates for each quadrant:
  x1 = sensor1; y1 = sensor1;
  x2 = -sensor2; y2 = sensor2;
  x3 = -sensor3; y3 = -sensor3;
  x4 = sensor4; y4 = -sensor4;

// Calculating the centres of mass for diagonals:
  centerX_diag1 = (x1 * sensor1 + x3 * sensor3) / (sensor1 + sensor3);
  centerY_diag1 = (y1 * sensor1 + y3 * sensor3) / (sensor1 + sensor3);

  centerX_diag2 = (x2 * sensor2 + x4 * sensor4) / (sensor2 + sensor4);
  centerY_diag2 = (y2 * sensor2 + y4 * sensor4) / (sensor2 + sensor4);

  centerX = abs((centerX_diag1 + centerX_diag2) / 2);
  centerY = abs((centerY_diag1 + centerY_diag2) / 2);

  centerX_values[filterIndex] = centerX;
  centerY_values[filterIndex] = centerY;

  // Update the index (ring buffer with 3 elements)
  filterIndex = (filterIndex + 1) % 3;

// Apply a median filter to centreX and centreY
  float filteredCenterX = medianFilter(centerX_values);
```

```
  float filteredCenterY = medianFilter(centerY_values);

  // Outputting filtered values to the serial port
  Serial.print(String(sensor2) + " ");
  Serial.print(String(sensor1) + " ");
  Serial.print(String(sensor3) + " ");
  Serial.print(String(sensor4) + " ");
  Serial.print(String(filteredCenterX) + " ");
  Serial.println(String(filteredCenterY));

  mySerial.print(String(sensor2) + "|");
  mySerial.print(String(sensor1) + "|");
  mySerial.print(String(sensor3) + "|");
  mySerial.print(String(sensor4) + "|");
  mySerial.print(String(filteredCenterX) + "|");
  mySerial.println(String(filteredCenterY));

  delay(100);

  // Resetting the counters for the next cycle
  counter = 0;
  sum0 = 0;
  sum1 = 0;
  sum2 = 0;
  sum3 = 0;
  }
}
```

APPENDIX C. Listing of software for personal computers

```
import controlP5.*;
ControlP5 cp5;

import javax.swing.JOptionPane;

int lastReminderTime;
int reminderInterval = 30*60 * 1000; // 30 minutes in milliseconds
int countdownStart ;

import processing.serial.*;
Serial port;

import processing.sound.*;
SoundFile sound;

boolean soundPlayed = false;

Chart myChart0;
Chart myChart1;
Chart myChart2;
Chart myChart3;

boolean pogana = false;

float CenterX;
float CenterY;

float CenterX0 = 0;
float CenterY0 = 0;

int A0 = 0;
int A1 = 0;
int A2 = 0;
int A3 = 0;

float size0=10;
float size1=10;
float size2=10;
float size3=10;

int colFrame=20;
boolean[] bufpogana = new boolean[colFrame];
int idGetData=0;
int persentTrue=50;

PFont VDSboldFont;
PFont VDSFont;

int lastSoundTime = 0;
```

```
int BUTTON_X = 715;
int BUTTON_Y = 80;
int BUTTON_WIDTH = 300;
int BUTTON_HEIGHT = 70;

boolean isButtonHovered = false;

String set_text;


void setup()
{

  print(Serial.list());
  port = new Serial(this, Serial.list()[0], 9600);
  port.bufferUntil('\n');

  sound = new SoundFile(this, "sound.mp3");

  size(1142, 624);
  background(255);

  cp5 = new ControlP5(this);

  VDSboldFont = createFont("D:/Documents/МАН/Шрифти/VDS_bold/VDS_Bold_New.ttf", 45);
  VDSFont = createFont("D:/Documents/МАН/Шрифти/VDS Thin/VDS_Thin.ttf", 25);

  textFont(VDSboldFont);

  String text_PO = "Posture position:";
  textSize(60);
  fill(0);
  text(text_PO, 30, 70);

  textFont(VDSFont);

  String text_1 = "Pressure on the 1-st sensor:";
  textSize(23);
  fill(0);
  text(text_1, 50, 180);

  String text_2 = "Pressure on the 2-nd sensor:";
  textSize(23);
  fill(0);
  text(text_2, 50, 290);

  String text_3 = "Pressure on the 3-rd sensor:";
  textSize(23);
  fill(0);
  text(text_3, 50, 400);
```

```
        String text_4 = "Pressure on the 4-th sensor:";
        textSize(23);
        fill(0);
        text(text_4, 50, 510);


        myChart0 = cp5.addChart("")
                .setPosition(50, 190)
                .setSize(500, 70)
                .setRange(0, 1023)
                .setView(Chart.LINE) // use Chart.LINE, Chart.PIE, Chart.AREA,
Chart.BAR_CENTERED
                .setColorCaptionLabel(color(50))
                ;
        myChart0.addDataSet("incoming");
        myChart0.setData("incoming", new float[100]);

        myChart1 = cp5.addChart(" ")
                .setPosition(50, 300)
                .setSize(500, 70)
                .setRange(0, 1023)
                .setView(Chart.LINE) // use Chart.LINE, Chart.PIE, Chart.AREA,
Chart.BAR_CENTERED
                .setColorCaptionLabel(color(50))
                ;
        myChart1.addDataSet("incoming");
        myChart1.setData("incoming", new float[100]);

        myChart2 = cp5.addChart("   ")
                .setPosition(50, 410)
                .setSize(500, 70)
                .setRange(0, 1023)
                .setView(Chart.LINE) // use Chart.LINE, Chart.PIE, Chart.AREA,
Chart.BAR_CENTERED
                .setColorCaptionLabel(color(50))
                ;
        myChart2.addDataSet("incoming");
        myChart2.setData("incoming", new float[100]);

        myChart3 = cp5.addChart("    ")
                .setPosition(50, 520)
                .setSize(500, 70)
                .setRange(0, 1023)
                .setView(Chart.LINE) // use Chart.LINE, Chart.PIE, Chart.AREA,
Chart.BAR_CENTERED
                .setColorCaptionLabel(color(50))
                ;
        myChart3.addDataSet("incoming");
        myChart3.setData("incoming", new float[100]);

        noFill();
        stroke(0);
```

```
  strokeWeight(10);
  rect(665, 190, 400, 400, 50);

 }

 void serialEvent(Serial port)
 {
  String input = port.readStringUntil('\n');

  if (input != null) {

  // The data is split into an array of Strings with a comma or asterisk as a delimiter and converted
into an array of integers.
  int[] vals = int(split(input, " "));
  float[] valsf = float(split(input, " "));


  // Fill r,g,b variables
  A0 = vals[0];
  A1 = vals[1];
  A2 = vals[2];
  A3 = vals[3];
  CenterX = valsf[4];
  CenterY = valsf[5];
  print(str(A0) + " ");
  print(str(A1) + " ");
  print(str(A2) + " ");
  print(str(A3) + " ");
  print(str(CenterX) + " ");
  println(str(CenterY) + " ");

  }
 }


 void draw()
 {

  textFont(VDSFont);

  noStroke();
  fill(255);
  rect(400, 160, 50, 30);
  textSize(10);
  fill(0);
  text(str(A0), 400, 180);

  noStroke();
  fill(255);
  rect(400, 270, 50, 30);
  textSize(10);
  fill(0);
```

```
text(str(A1), 400, 290);

noStroke();
fill(255);
rect(400, 380, 50, 30);
textSize(10);
fill(0);
text(str(A2), 400, 400);


noStroke();
fill(255);
rect(400, 490, 50, 30);
textSize(10);
fill(0);
text(str(A3), 400, 510);

//A0
fill(255);
stroke(70);
strokeWeight(7);
ellipse(750, 278, 60, 60);

fill(#5C62F0);
noStroke();
size0 = map(A0, 0, 1024, 10, 55);
ellipse(750, 278, size0, size0);

//A1
fill(255);
stroke(70);
strokeWeight(7);
ellipse(980, 278, 60, 60);

fill(#5C62F0);
noStroke();
size1 = map(A1, 0, 1024, 10, 55);
ellipse(980, 278, size1, size1);

//A2
fill(255);
stroke(70);
strokeWeight(7);
ellipse(775, 502, 60, 60);

fill(#5C62F0);
noStroke();
size2 = map(A2, 0, 1024, 10, 50);
ellipse(775, 502, size2, size2);

//A3
fill(255);
```

```
stroke(70);
strokeWeight(7);
ellipse(955, 502, 60, 60);

fill(#5C62F0);
noStroke();
size3 = map(A3, 0, 1024, 10, 50);
ellipse(955, 502, size3, size3);

//Charts
myChart0.push("incoming", A0);

myChart1.push("incoming", A1);

myChart2.push("incoming", A2);

myChart3.push("incoming", A3);

//BUTTON
fill(225);
noStroke();

if (mouseX >= BUTTON_X && mouseX <= BUTTON_X + BUTTON_WIDTH && mouseY >=
BUTTON_Y && mouseY <= BUTTON_Y + BUTTON_HEIGHT)
{
isButtonHovered = true;
fill(#8296DB);
}
else
{
isButtonHovered = false;
fill(205);
}

if(mousePressed && isButtonHovered)
{
CenterX0 = CenterX;
CenterY0 = CenterY;
println("Calibrated");
for (int i =0;i<colFrame;i++){
bufpogana[i]=false;

}
}

rect(BUTTON_X, BUTTON_Y, BUTTON_WIDTH, BUTTON_HEIGHT, 10);


textFont(VDSboldFont);
String text_kn = "CALIBRATE";
textSize(45);
fill(0);
```

```
text(text_kn, 744, 130);

fill(255);
noStroke();
rect(40, 80, 650, 80);


if(CenterX0==0  &&  CenterY0==0)
{
 set_text = "Calibrate the seat";
 textSize(65);
 fill(#8084DB);
}
else
{


 if(A0>=0 && A0<=10  &&  A1>=0 && A1<=10   &&  A2>=0 && A2<=10  &&  A3>=0 &&
A3<=10)
 {

        set_text = "Sit on the seat";
        textSize(65);
        fill(120);
 }
        else
 {
        if(CenterX>=CenterX0-15 && CenterX<=CenterX0+15 && CenterY>=CenterY0-15 &&
CenterY<=CenterY0+15)
        {
        bufpogana[idGetData]=false;
        idGetData++;
        idGetData%=colFrame;

        }
        else
        {
        bufpogana[idGetData]=true;
        idGetData++;
        idGetData%=colFrame;


        }
        int col=0;
        for (int i =0;i<colFrame;i++){
        if(bufpogana[i]){
        col++;
        }
        }
        if(col>=colFrame/2){
        set_text = "Sit up straighter on the seat :|";
        textSize(50);
```

```
            fill(#ED84A0);

            }
            else {
            set_text = "Good posture :)";
            textSize(65);
            fill(#84ED94);
            }
  }
  }




  text(set_text, 40, 140);



  if (millis() - lastReminderTime > reminderInterval) {
  countdownStart = millis(); // Reset the countdown start time
  lastReminderTime = millis(); // Reset the timer

  String message = "Time for a 5-minute warm-up!\n\n";
  JOptionPane.showMessageDialog(null, message); // Show pop-up reminder with countdown
  }

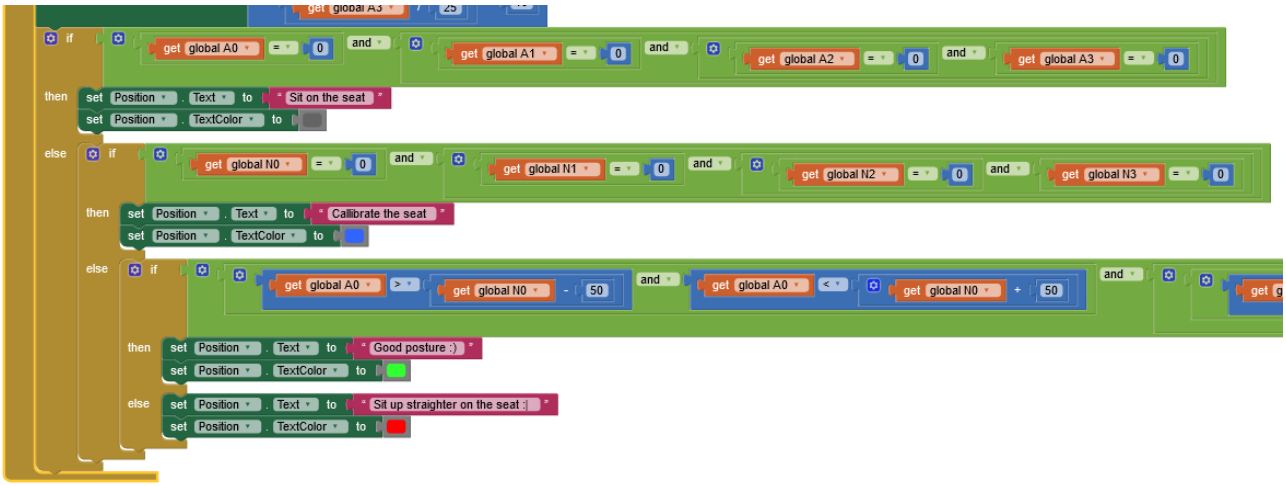
}
```

APPENDIX D.

Software for mobile devices implemented in MIT App Inventor






if
get global A0 = 0 and get global A1 = 0 and get global A2 = 0 and get global A3 = 0
then
set Position . Text to " Sit on the seat "
set Position . TextColor to
else if
get global N0 = 0 and get global N1 = 0 and get global N2 = 0 and get global N3 = 0
then
set Position . Text to " Callibrate the seat "
set Position . TextColor to
else if
get global A0 > get global N0 - 50 and get global A0 < get global N0 + 50
then
set Position . Text to " Good posture :) "
set Position . TextColor to
else
set Position . Text to " Sit up straighter on the seat : "
set Position . TextColor to


and
get global A3 > get global A3 - 30
and
get global A3 < get global A3 + 30

APPENDIX E.

3-D and 2-D view in the EASYEDA environment

